Oxford Cambridge and RSA – style practice paper

GCSE (9–1) Computer Science

J277/02 Computational thinking, algorithms and programming

Topic 2.2: Programming fundamentals

Candidate forename		**Candidate surname**	
Centre number		**Candidate number**	

Practice questions in the style of OCR GCSE Computer Science (J277). Every question is original; the layout, command words, mark allocations and mark scheme follow the real exam format.

Time allowed	About 120 minutes (roughly 1 minute per mark)
Instructions	• Use black ink. HB pencil may be used for graphs and diagrams only. • Answer **all** the questions. • Write your answer to each question in the space provided. • Where the question asks you to write code or pseudocode, you may use any suitable high-level language or the OCR Reference Language; marks are for the logic, not exact syntax. • Do **not** use a calculator or any other electronic device unless told to.
Information	• The total mark for this paper is **110**. • The marks for each question are shown in brackets [ ]. • Quality of extended responses will be assessed in questions marked with an asterisk (*). • Where a question asks you to *state* or *give*, a short answer is enough; *describe* and *explain* need more detail.

Name: _____ Date: _____ Mark: _____ / 110

1 A program stores different types of data.

(a) Draw **one** line from each data item to its correct data type. **[4]**

Data item	Data type
42	Boolean
3.14	String
True	Integer
"Hello"	Real

(b) State what is meant by a **variable**. **[1]**

(c) State **one** reason for using a **constant** instead of a variable for the value of VAT (20%). **[2]**

(d) The code below asks for the user's age. The input is a string. Explain why the programmer converts it using int(). **[2]**

```
age = int(input("Enter your age"))
if age >= 18 then
    print("Adult")
endif
```

2 The code shows arithmetic operations in the OCR Reference Language.

(a) State the value stored in each variable after the code has run. **[6]**

```
a = 17 MOD 5
b = 17 DIV 5
c = 2 ^ 4
d = 7 + 3 * 2
e = (7 + 3) * 2
f = 9 / 2
```

Variable	Value
a	
b	
c	
d	
e	
f	

(b) Give **one** use of the **MOD** operator in a program. **[1]**

(c) Write a line of code that stores **True** in a variable called **even** if the integer **n** is even. **[2]**

3 A program uses comparison and Boolean operators. The variables are x = 8 and y = 2.

(a) State whether each expression evaluates to **True** or **False**. **[5]**

Expression	True or False?
x > 5 AND y > 5	
x > 5 OR y > 5	
NOT (x == 8)	
x MOD 2 == 0	
x != y	

(b) State the difference between the operators **=** and **==** in the OCR Reference Language. **[2]**

4 A teacher uses this program to convert marks into grades.

(a) State the output when the user enters **72**. **[1]**

```
mark = int(input("Enter mark"))
if mark >= 80 then
    print("Distinction")
elseif mark >= 60 then
    print("Merit")
elseif mark >= 40 then
    print("Pass")
else
    print("Fail")
endif
```

(b) State the output when the user enters **39**. **[1]**

(c) A student enters 80. Explain why the program prints Distinction and not Merit, even though 80 is also greater than 60. **[2]**

(d) The teacher wants the program to also print "Invalid" if the mark is above 100 or below 0. Write the additional condition. **[2]**

(e) Give **one** test value that checks the boundary between Pass and Merit. **[1]**

5 A program manipulates strings. The variable **word** contains the string "Computer".

(a) State the value of each expression. **[4]**

Expression	Value
word.length	
word.left(3)	
word.substring(3, 4)	
word.upper	

(b) The ASCII code for the letter **A** is 65. Write the output of the code below. **[2]**

```
letter = "A"
code = ASC(letter) + 2
print(CHR(code))
```

(c) Write code that asks the user to enter their first name and then prints the first letter of the name in upper case. **[3]**

(d) Explain what is meant by **concatenation** and give an example. **[2]**

6 Iteration is used to repeat code.

(a) State the output of the following program. **[2]**

```
for i = 0 to 10 step 2
    print(i)
next i
```

(b) State how many times the word **Hi** is printed. **[1]**

```
for a = 1 to 3
    for b = 1 to 2
        print("Hi")
    next b
next a
```

(c) State the output of the code below. **[1]**

```
x = 100
count = 0
while x > 10
    x = x / 2
    count = count + 1
endwhile
print(count)
```

(d) State the output of the code below. **[1]**

```
n = 1
do
    n = n * 3
until n > 50
print(n)
```

(e) Explain the difference between a **while** loop and a **do ... until** loop. **[2]**

(f) State what is meant by an **infinite loop**. **[1]**

7 Arrays store several values under one name.

(a) State the value of **scores[3]**. The first element is at index 0. **[1]**

```
array scores = [12, 7, 19, 4, 15]
```

(b) Write the output of the code below. **[2]**

```
array scores = [12, 7, 19, 4, 15]
total = 0
for i = 0 to 4
    total = total + scores[i]
next i
print(total)
```

(c) Write code to find and print the **largest** value in the array **scores**. **[4]**

(d) The 2D array **grid** stores the values in the table. State the values of **grid[1,2]** and **grid[2,0]**. The first row and column are 0. **[2]**

	column 0	**column 1**	**column 2**
row 0	5	8	2
row 1	9	1	7
row 2	3	6	4

(e) Describe **one** advantage of using an array instead of separate variables to store 30 test scores. **[2]**

8 Subprograms break programs into smaller parts.

(a) State **one** difference between a function and a procedure. **[1]**

(b) State the output of the code below. **[1]**

```
function triple(n)
    result = n * 3
    return result
endfunction

x = triple(4)
print(x)
```

(c) Identify the **parameter** in the function above. **[1]**

(d) The line print(result) is added at the end of the program. Explain why it causes an error. **[2]**

(e) Write a function called **area** that takes the length and width of a rectangle as parameters and returns the area. **[3]**

(f) Give **two** advantages of using subprograms. **[2]**

9 Files and databases.

(a) The program reads a text file called scores.txt. Describe what the code does. **[3]**

```
myFile = open("scores.txt")
while NOT myFile.endOfFile()
    line = myFile.readLine()
    print(line)
endwhile
myFile.close()
```

(b) State why a file should be closed after use. **[1]**

(c) The table **Students** is stored in a database. State the output of this SQL query. **[2]**

```
SELECT Name FROM Students
WHERE Year = 10 AND Score > 50
```

ID	Name	Year	Score
1	Amir	10	68
2	Beth	11	54
3	Chen	10	91
4	Dara	11	77
5	Eve	10	45

(d) Write an SQL query to display the name and score of all students in Year 11. **[3]**

10 Writing programs.

(a) A game simulates throwing two dice. The function random(1, 6) returns a random integer from 1 to 6 inclusive. Write a program that throws two dice and outputs the total. If both dice show the same number it should also output "Double". **[5]**

(b) A program asks the user to enter a password. The password must be at least 8 characters long. Write code that repeats the request until a valid password is entered. **[4]**

(c) Write a program that prints the 7 times table from 1 × 7 to 12 × 7 in the format **3 x 7 = 21**. **[4]**

11 Extended response questions.

(a) *A school wants a program to store and process the exam marks for 200 students. Discuss how the programmer could use arrays, loops and subprograms to make the program efficient and easy to maintain. **[8]**

(b) *Compare the use of for, while and do ... until loops. In your answer, give an example of when each would be most suitable. **[8]**

END OF QUESTIONS

MARK SCHEME – Topic 2.2: Programming fundamentals

How to use this mark scheme: mark your own work in a different colour. A bullet (•) in the Answer column is a separate marking point worth one mark unless stated. A slash (/) separates alternative answers that are equally acceptable. Words in brackets are not needed. **Accept** = acceptable; **Ignore** = neither credited nor penalised; **Do not accept** = contradicts or spoils the answer. For programming questions, any valid solution that works is acceptable and syntax errors are not penalised if the intent is clear. The code in the answers uses the OCR Reference Language style.

Question	Answer	Marks	Guidance
1(a)	• 42: integer ; • 3.14: real ; • True: Boolean ; • "Hello": string ;	4 AO1 2.2.1	One mark per correct match.
1(b)	a named memory location that stores a value that can change during the program ;	1 AO1 2.2.1	
1(c)	• value cannot be changed accidentally during the program ; • only needs to be changed in one place if the rate changes / improves readability ;	2 AO2 2.2.1	
1(d)	• input() returns a string, which cannot be used in calculations / comparisons as a number ; • int() casts (converts) it to an integer ;	2 AO2 2.2.1	
2(a)	• a = 2 ; • b = 3 ; • c = 16 ; • d = 13 ; • e = 20 ; • f = 4.5 ;	6 AO2 2.2.2	
2(b)	• to test if a number is even / odd ; • to find the remainder after division / wrap around ;	1 AO1 2.2.2	Any one.
2(c)	• use of n MOD 2 ; • even = (n MOD 2 == 0) / if statement setting even to True ;	2 AO3 2.2.2	
3(a)	• x > 5 AND y > 5: False ; • x > 5 OR y > 5: True ; • NOT (x == 8): False ; • x MOD 2 == 0: True ; • x != y: True ;	5 AO2 2.2.2	
3(b)	• = assigns a value to a variable ; • == compares two values for equality (returns True / False) ;	2 AO1 2.2.2	
4(a)	Merit ;	1 AO2 2.2.3	
4(b)	Fail ;	1 AO2 2.2.3	
4(c)	• the conditions are checked in order, top to bottom ; • the first true condition (mark >= 80) is run and the rest are skipped ;	2 AO3 2.2.3	
4(d)	• mark > 100 OR mark < 0 ; • if ... then print("Invalid") structure ;	2 AO3 2.2.3	
4(e)	59 or 60 ;	1 AO3 2.2.3	

Question	Answer	Marks	Guidance
5(a)	• 8 ; • Com ; • pute ; • COMPUTER ;	4 AO2 2.2.4	The first character is at position 0. Accept: left(3) = 'Com'.
5(b)	• ASC("A") + 2 = 67 ; • CHR(67) = C ;	2 AO2 2.2.4	
5(c)	• input stored in a variable ; • extract first character (name.left(1) / substring(0,1)) ; • convert to upper case and print ;	3 AO3 2.2.4	
5(d)	• joining two (or more) strings together ; • e.g. "Hello " + name ;	2 AO1 2.2.4	
6(a)	• 0 2 4 6 8 10 (each printed on a new line) ; • (allow 1 mark for 5 or 6 values with one error) ;	2 AO2 2.2.5	
6(b)	6 ;	1 AO2 2.2.5	
6(c)	4 ;	1 AO2 2.2.5	x: 100, 50, 25, 12.5, 6.25.
6(d)	81 ;	1 AO2 2.2.5	
6(e)	• while checks the condition before the code in the loop is run (may never run) ; • do until checks after, so the code runs at least once ;	2 AO1 2.2.5	
6(f)	a loop that never stops because the condition is never met / false ;	1 AO1 2.2.5	
7(a)	4 ;	1 AO2 2.2.6	
7(b)	• 57 ; • (method mark: running total 12, 19, 38, 42, 57) ;	2 AO2 2.2.6	
7(c)	• largest initialised to scores[0] ; • loop through all elements ; • if scores[i] > largest then largest = scores[i] ; • print largest ;	4 AO3 2.2.6	
7(d)	• 7 ; • 3 ;	2 AO2 2.2.6	
7(e)	• can use a loop to process all of the scores ; • less code / easier to maintain / one name for all the data ;	2 AO2 2.2.6	
8(a)	a function returns a value; a procedure does not ;	1 AO1 2.2.7	
8(b)	12 ;	1 AO2 2.2.7	
8(c)	n ;	1 AO2 2.2.7	
8(d)	• result is a local variable ; • it only exists inside the function so cannot be used outside ;	2 AO3 2.2.7	
8(e)	• function area(length, width) (name and parameters) ; • calculates length * width ; • return and endfunction ;	3 AO3 2.2.7	

Question	Answer	Marks	Guidance
8(f)	• code can be reused / written once and called many times ; • easier to test / debug / maintain ; • program is shorter / easier to read ;	2 AO1 2.2.7	Any two.
9(a)	• opens the file ; • reads and prints each line one at a time until the end of the file ; • closes the file ;	3 AO2 2.2.8	
9(b)	so the data is saved / to free the file for other programs / avoid corruption ;	1 AO1 2.2.8	
9(c)	• Amir ; • Chen ;	2 AO2 2.2.8	
9(d)	• SELECT Name, Score ; • FROM Students ; • WHERE Year = 11 ;	3 AO3 2.2.8	
10(a)	• two random numbers generated using random(1,6) ; • total calculated ; • total output ; • comparison of the two dice ; • "Double" output when equal ;	5 AO3 2.2.9	
10(b)	• input inside a loop ; • check length >= 8 using .length ; • loop continues while the password is too short ; • message / loop ends when valid ;	4 AO3 2.2.9	
10(c)	• loop from 1 to 12 ; • multiplication i * 7 ; • output in the correct format with concatenation ; • loop ended correctly (next / endfor) ;	4 AO3 2.2.9	

Question	Answer	Marks	Guidance
11(a)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • array stores all 200 marks under one name with an index instead of 200 variables • a loop can process every element (e.g. total, average, find highest / lowest) with a few lines of code • 2D array could store marks for several subjects per student • subprograms: separate functions e.g. calculateAverage(), findHighest(), displayGrade(), each doing one job • subprograms can be reused with different data and tested independently • parameters and return values pass data in and out; local variables avoid errors • the program is shorter, easier to read, debug and modify • balanced conclusion / possible disadvantage: fixed size array, more planning needed	8 AO1 / AO2 / AO3 2.2.6 / 2.2.7	
11(b)	**Level 3 (high), 6–8 marks**: Thorough knowledge and understanding of the topic with detailed, accurate and relevant points. A balanced, well-structured answer reaching a justified conclusion where asked. Technical terms used correctly throughout. **Level 2 (mid), 3–5 marks**: Reasonable knowledge and understanding. Some relevant points made, but the answer lacks detail, balance or a clear structure in places. **Level 1 (low), 1–2 marks**: Basic knowledge. A few simple, mostly isolated points with little or no development or linking. **Level 0, 0 marks**: No response or no response worthy of credit. **Indicative content** (credit any other valid points): • for loop: count-controlled; repeats a set number of times; e.g. processing every item in an array of 30 scores • while loop: condition-controlled; condition checked before the loop starts, so the code may never run; e.g. reading a file until the end • do ... until: condition-controlled; condition checked at the end so the loop runs at least once; e.g. asking for a password until it is valid • for loop is simple but cannot easily stop early without extra code; counter is controlled automatically • while / do until risk infinite loops if the condition is never changed • all three can often solve the same problem but choosing the right one makes code clearer • balanced conclusion linking each to a suitable example	8 AO1 / AO2 / AO3 2.2.5	

Where marks are lost in Topic 2.2

Use this page after marking. Tick the ones that cost you marks, then redo those questions closed-book.

- ☐ **Trace code line by line.** Write the value of every variable after each line — it is the only reliable way to get output questions right.
- ☐ **DIV and MOD:** 17 DIV 5 = 3 (whole number of times); 17 MOD 5 = 2 (remainder). MOD 2 == 0 tests for even numbers.
- ☐ **Array indexes start at 0.** An array of 5 values has indexes 0–4; going to 5 causes an error.
- ☐ **Initialise before looping:** total = 0, largest = first item. Forgetting this loses marks every year.
- ☐ **Local vs global:** a variable made inside a subprogram cannot be used outside it.
- ☐ **Function vs procedure:** a function RETURNS a value; a procedure does not.
- ☐ **= vs ==:** = assigns, == compares. Writing = inside an if condition is a common error.
- ☐ **Write your own code in the exam** using OCR Reference Language or a language you know; indent so the structure is clear and keep to the question's required inputs / outputs.